

1.2 - Esercitazione specifiche algebriche

Esercizio 1

Dimostrare che la seguente affermazione è vera:

$$\text{append}(\text{new}(), \text{add}(\text{new}(), c)) = \text{add}(\text{new}(), c)$$

Occorre utilizzare 2 regole fondamentali:

- **Regola ricorsiva dell'append:** $\text{append}(s1, \text{add}(s2, c)) = \text{add}(\text{append}(s1, s2), c)$. Questa regola ci dice come comportarci quando concateniamo una stringa a un'altra stringa a cui è stato aggiunto un carattere.
- **Caso base dell'append:** $\text{append}(s, \text{new}()) = s$. Questa regola indica che concatenare una stringa vuota ($\text{new}()$) a una stringa s non modifica la stringa s .

Dimostrare che:

$$\text{append}(\text{new}(), \text{add}(\text{new}(), c)) = \text{add}(\text{new}(), c)$$

$$\Rightarrow \text{utilizzando } \text{append}(s1, \text{add}(s2, c)) = \text{add}(\text{append}(s1, s2), c)$$

$$\text{add}(\text{append}(\text{new}(), \text{new}()), c) = \text{add}(\text{new}(), c)$$

$$\Rightarrow \text{utilizzando } \text{append}(s, \text{new}()) = s \quad (s = \text{new}() \Rightarrow \text{append}(\text{new}(), \text{new}()) = \text{new}())$$

$$\text{add}(\text{new}(), c) = \text{add}(\text{new}(), c)$$

Procedura dell'esercizio:

- **Passaggio 1:** Prendi in considerazione la parte sinistra dell'equazione da dimostrare: $\text{append}(\text{new}(), \text{add}(\text{new}(), c))$.
- **Passaggio 2:** Applica la **Regola 1** all'espressione. In questo contesto specifico, la variabile $s1$ corrisponde a $\text{new}()$, la variabile $s2$ corrisponde a $\text{new}()$ e la variabile del carattere rimane c .
- **Passaggio 3:** Riscrivendo l'espressione in base alla Regola 1, ottieni: $\text{add}(\text{append}(\text{new}(), \text{new}()), c)$.
- **Passaggio 4:** Ora devi semplificare la parte più interna dell'espressione, ovvero $\text{append}(\text{new}(), \text{new}())$. Per farlo, utilizza la **Regola 2**.
- **Passaggio 5:** Considerando che se $s = \text{new}()$, l'applicazione della Regola 2 ci porta a dedurre che $\text{append}(\text{new}(), \text{new}()) = \text{new}()$.
- **Passaggio 6:** Sostituisci questo risultato all'interno dell'espressione trovata al Passaggio 3. L'espressione $\text{add}(\text{append}(\text{new}(), \text{new}()), c)$ diventa quindi semplicemente $\text{add}(\text{new}(), c)$.

Esercizio 2

Definire la specifica algebrica della struttura dati `Coord` che rappresenta una coordinata nello spazio cartesiano a due dimensioni.

sorts: `Coord`, `Integer`, `Boolean`

operations:

`Create(Integer, Integer) → Coord;`

`X(Coord) → Integer ;`

`Y(Coord) → Integer;`

`Eq(Coord, Coord) → Boolean;`

Sort i tipi: ci servono `Coord` (il nostro nuovo tipo), `Integer` (per i valori delle coordinate) e `Boolean` (per i test di uguaglianza).

Operation funzioni:

- `Create(Integer, Integer) → Coord;` È il costruttore, prende due numeri interi (la `x` e la `y`) e genera la coordinata.
- `X(Coord) → Integer ; Y(Coord) → Integer;` È un osservatore, prende una coordinata e ci restituisce il suo valore nell'asse `x`.
- `Y(Coord) → Integer;` È un osservatore, prende una coordinata e ci restituisce il suo valore nell'asse `y`.
- `Eq(Coord, Coord) → Boolean;` È un osservatore, prende le due coordinate e ci dice se sono uguali (restituisce un valore di verità vero o falso).

Osservatori	Costruttori di <code>c'</code>
	<code>Create(x,y)</code>
<code>X(c')</code>	<code>x</code>
<code>Y(c')</code>	<code>y</code>
<code>Eq(c',c)</code>	if <code>x=X(c)</code> then if <code>y= Y(c)</code> then <code>true</code> else <code>false</code> else <code>false</code>

Estrazione della `x`

`X(Create(x, y)) = x`

Se creo una coordinata usando i valori `x` e `y`, e poi chiedo di osservare `X`, il risultato deve essere esattamente `x`.

Estrazione della `Y`

`Y(Create(x, y)) = y`

Se creo una coordinata usando i valori `x` e `y`, e poi chiedo di osservare `Y`, il risultato deve essere esattamente `y`.

Verifica di uguaglianza

```
Eq(Create(x, y), c) = if x = X(c) then (if y = Y(c) then true else false)
else false
```

Se creo una coordinata usando i valori x e y , e poi chiedo di verificare se è uguale a un'altra coordinata generica c , il sistema controlla prima se la mia x è uguale alla componente X di c . Se lo è, controlla se anche la mia y è uguale alla componente Y di c .

Se entrambe le condizioni sono soddisfatte, il risultato è vero (`true`); in qualsiasi altro caso, è falso (`false`).

Esercizio 3

Realizzare la specifica algebrica della struttura dati albero binario utilizzando il seguente ordine di inserimento: se il nodo da inserire è minore del nodo corrente inserirlo nel sotto albero sinistro altrimenti inserirlo nel sotto albero destro. L'albero binario contiene dati generici di tipo Elem.

Osservatori	Costruttori di t'	
	Create ()	if e' < e then Add(l, e') else Add(r, e')
Add(t', e')	Build(t', e', t')	if e' < e then Add(l, e') else Add(r, e')
Left(t')	Create()	l
Right(t')	Create()	r
Data(t')	error	e
IsEmpty(t')	true	false
Contains(t', e')	false	if e'=e then true else if (e'<e then Contains(l, e') else Contains(r, e')

Qui abbiamo 2 costruttori:

- `Create()` : crea un albero vuoto.
- `Build(l, e, r)` : costruisce un albero partendo da un sottoalbero sinistro l , un elemento radice e e un sottoalbero destro r .

Poiché abbiamo due costruttori, per quasi ogni operazione (osservatore) dovremo scrivere due formule: una che spiega cosa succede se l'albero è vuoto, e una che spiega cosa succede se l'albero contiene già dei nodi.

Inserimento di un elemento

```
Add(Create(), e') = Build(Create(), e', Create())
```

Se provo ad aggiungere un elemento e' a un albero vuoto, il risultato è un nuovo albero (costruito con `Build`) che ha e' come radice e due sottoalberi vuoti (destro e sinistro).

Inserimento in un albero non vuoto

```
Add(Build(l, e, r), e') = if e' < e then Add(l, e') else Add(r, e')
```

Se provo ad aggiungere un elemento e' a un albero già formato (con radice e), il sistema confronta i due valori. Se il nuovo elemento e' è minore della radice e , l'elemento viene inserito (ricorsivamente) nel sottoalbero sinistro l . Altrimenti, viene inserito nel sottoalbero destro r .

Estrazione da un albero vuoto

```
Left(Create()) = Create()
```

Se chiedo il sottoalbero sinistro di un albero vuoto, il risultato è semplicemente un albero vuoto.

Estrazione da un albero non vuoto

```
Left(Build(l, e, r)) = l
```

Se ho un albero costruito con un lato sinistro l , una radice e e un lato destro, e chiedo di osservare il suo lato sinistro, il risultato è esattamente l .

Estrazione da un albero vuoto

```
Right(Create()) = Create()
```

Se chiedo il sottoalbero destro di un albero vuoto, il risultato è un albero vuoto.

Estrazione da un albero non vuoto

```
Right(Build(l, e, r)) = r
```

Se ho un albero costruito e chiedo di osservare il suo lato destro, il risultato è esattamente il sottoalbero destro r .

Lettura da un albero vuoto

```
Data(Create()) = error
```

Se cerco di leggere il valore contenuto nel nodo radice di un albero che è vuoto, il sistema mi restituisce un errore (perché non c'è nessun dato da leggere).

Lettura da un albero non vuoto

```
Data(Build(l, e, r)) = e
```

Se chiedo di leggere il dato della radice di un albero già costruito, il risultato è esattamente l'elemento e .

Verifica su albero vuoto

```
IsEmpty(Create()) = true
```

Se chiedo se un albero appena creato con `Create()` (quindi vuoto) è effettivamente vuoto, la risposta è ovviamente vero (`true`).

Verifica su albero non vuoto

```
IsEmpty(Build(l, e, r)) = false
```

Se chiedo se un albero costruito con dei nodi (`Build`) è vuoto, la risposta è falso (`false`).

Ricerca in un albero vuoto

```
Contains(Create(), e') = false
```

Se cerco un qualsiasi elemento e' all'interno di un albero vuoto, il risultato è falso (non può esserci).

Ricerca in un albero non vuoto

```
Contains(Build(l, e, r), e') = if e' = e then true else (if e' < e then
Contains(l, e') else Contains(r, e'))
```

Se cerco un elemento e' in un albero che ha come radice e , controllo prima se e' è uguale a e . Se lo è, ho trovato l'elemento (`true`). Se non lo è, controllo se e' è minore di e : se è minore, continuo la ricerca nel sottoalbero sinistro l , altrimenti continuo la ricerca nel sottoalbero destro r .

Esercizio 4

Definire la specifica algebrica del tipo `Cursore`.

Un `Cursore` è una rappresentazione di una posizione sullo schermo. Le operazioni definite sono:

- `Create`: associa ad un cursore un'icona in una specifica posizione dello schermo
- `Position`: restituisce la posizione corrente del cursore
- `Translate`: sposta il cursore andando a modificare le coordinate attuali di una certa quantità
- `Change_Icon`: che permette di cambiare l'icona associata al cursore

N.B. Come le immagini sono create sullo schermo e come viene associata un'immagine (icona) al cursore sono dettagli implementativi che non inseriamo nella specifica.

Qui riutilizziamo il tipo `Coord` unendolo a un nuovo tipo `Bitmap`, il tipo `Bitmap` rappresenta un'immagine su schermo, nel nostro caso l'icona.

Il costruttore unico in questo caso è `Create(d, b)`, dove d è la coordinata e b è la bitmap (l'icona).

Lettura della Posizione (`Position`)

```
Position(Create(d, b)) = d
```

Se creo un cursore associandogli una coordinata d e un'icona b , e poi chiedo di osservare la sua posizione, il risultato deve essere esattamente la coordinata d originale.

Spostamento del Cursore (`Translate`)

```
Translate(Create(d, b), xi, yi) = Create(Coord.Create(Coord.X(d) + xi,
Coord.Y(d) + yi), b)
```

Se prendo un cursore (creato alla posizione d con l'icona b) e chiedo di spostarlo di una quantità xi sull'asse delle ascisse e yi sull'asse delle ordinate, il risultato è un **nuovo cursore**. Questo nuovo cursore manterrà la stessa identica icona b , ma avrà una nuova

posizione calcolata prendendo la X originale di d e sommandole x_i , e prendendo la Y originale di d e sommandole y_i .

(Nota: per creare questa nuova posizione usiamo le operazioni della specifica Coord che abbiamo costruito nell'Esercizio 2).

Cambio dell'Icona (`Change_Icon`)

```
Change_Icon(Create(d, b), b') = Create(d, b')
```

Se ho un cursore alla posizione d con l'icona b , e chiedo di cambiare la sua icona con una nuova immagine b' , il sistema mi restituisce un **nuovo cursore** che si trova nell'esatta posizione d in cui si trovava prima, ma a cui è associata la nuova icona b' .